

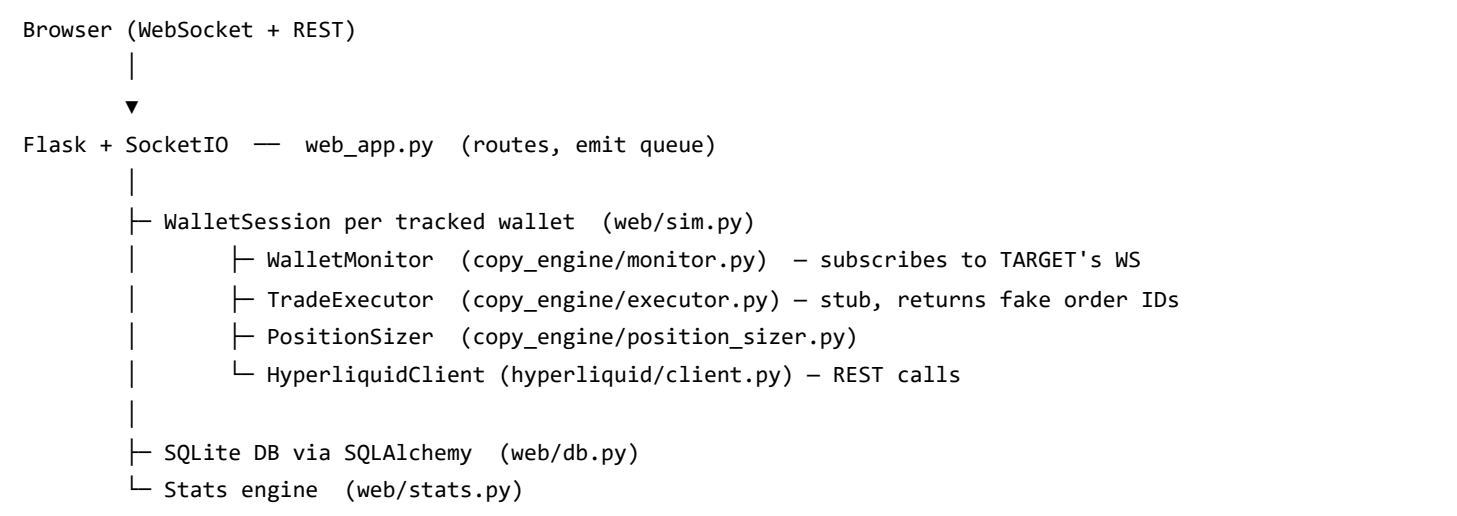
Hyperliquid Copy-Bot — Complete Logic Reference

This document describes every piece of logic in the simulation engine: how it works, why it works that way, and what the exact formulas are. It covers the full stack from WebSocket events to the database.

Table of Contents

1. [Architecture Overview](#)
2. [Configuration System](#)
3. [Database Schema](#)
4. [Session Lifecycle](#)
5. [Trading Style Auto-Detection](#)
6. [Startup Seeding — Ghost Positions & Decision Engine](#)
7. [Fill Processing Pipeline](#)
8. [Simulation Accuracy Models](#)
9. [Risk Management Guards](#)
10. [HFT Debounce System](#)
11. [Leverage Change Tracking](#)
12. [Periodic Tasks](#)
13. [Analytics & Stats Engine](#)
14. [REST API Endpoints](#)
15. [HFT Calibration System](#)
16. [Accuracy & Production Readiness](#)

1. Architecture Overview



Threading model

Flask runs in the main thread. All asyncio tasks (WebSocket listeners, periodic snapshots, debounce timers) run in a single background thread that owns a persistent `asyncio.EventLoop`. Communication between Flask routes and the async world goes through `submit(coro)` which calls `asyncio.run_coroutine_threadsafe`.

Emitted SocketIO events are queued into a thread-safe `queue.Queue` and drained by a Flask-SocketIO background task (`_emit_worker`), avoiding cross-thread emit races.

Session registry

`_sessions: dict[address → WalletSession]` is the in-memory registry. All Flask routes and async callbacks look up sessions here. It is keyed by lowercase Ethereum address.

2. Configuration System

All config lives in `config/settings.py` as Pydantic `BaseModel` classes. A single `settings` singleton is loaded once at module import. Environment variables override defaults; no hot-reload.

Config sections

Class	Attached as	Purpose
HyperliquidConfig	settings.hyperliquid	API / WS URLs, wallet address, private key
SizingConfig	settings.sizing	Position size mode (proportional / fixed), caps
LeverageConfig	settings.leverage	Leverage adjustment ratio, global min/max

Class	Attached as	Purpose
CopyRulesConfig	settings.copy_rules	Blocked assets, dust guard, max open trades
RiskManagementConfig	settings.risk_management	Daily loss cap, circuit breaker, net exposure cap
StartupSeedingPolicy	settings.seed_policy	Startup seeding thresholds (drift, exposure, leverage)
CopyStyleConfig	settings.copy_style	HFT detection threshold, default debounce seconds
SimAccuracyConfig	settings.sim_accuracy	Slippage bps, latency ms, maker close rate

Key defaults

taker_fee_rate	= 0.00045	(4.5 bps, HL Tier 0)
slippage_bps	= 3.0	(3 bps per side)
sim_latency_ms	= 150	(150 ms execution delay)
hft_threshold_fills_per_hour	= 60	(above this → debounced mode)
hft_debounce_secs	= 30	(initial debounce; overridden by adaptive logic)
max_seed_drift_pct	= 0.015	(1.5% drift triggers GHOST_ONLY at startup)
max_seed_leverage	= 4	(leverage cap for seeded positions)
startup_seed_size_multiplier	= 0.35	(35% of normal copy size at startup)
fast_loss_pct	= 0.05	(5% equity drop in window → circuit breaker)
fast_loss_window_secs	= 300	(5-minute rolling window)
max_net_exposure_pct	= 0.80	(80% long-short imbalance cap)

3. Database Schema

SQLite with WAL mode (write-ahead logging prevents corruption on crash). All tables are created by `Base.metadata.create_all()` at import time. New columns are added via `ALTER TABLE` migrations at the bottom of `db.py` — each migration is wrapped in a `try/except` that ignores "duplicate column" errors, making the schema upgrade idempotent across restarts.

Tables

wallets

One row per tracked wallet. Survives restarts.

Column	Type	Notes
address	String PK	Lowercase Ethereum address
label	String	Display name
start_balance	Float	Initial simulated balance

Column	Type	Notes
copy_mode	String	"all_fills" or "debounced" — auto-set by style detection
debounce_secs	Integer	Adaptive debounce window in seconds
detected_style	String	"Swing" or "HFT" — UI badge
created_at	DateTime	Registration timestamp

simulated_positions

Open positions. One row per (wallet, symbol). Deleted on close.

Column	Type	Notes
wallet_addr	String	FK to wallets
symbol	String	e.g. "BTC"
side	String	"LONG" or "SHORT"
size	Float	Absolute coin quantity
entry_price	Float	VWAP entry price
leverage	Integer	Current leverage (updated on leverage-change events)
margin_used	Float	Locked collateral = $\text{size} \times \text{entry} / \text{leverage}$
copy_ratio	Float	Ratio locked at position-open time
updated_at	DateTime	Last upsert time

ghost_positions

Positions the bot is aware of but chose not to open (failed startup seeding checks). Events for ghost symbols are silently absorbed; no orders are ever generated.

Column	Type	Notes
wallet_addr	String	FK to wallets
symbol	String	
side	String	LONG / SHORT (target's side)
target_size	Float	Updated on every event from target
target_entry_price	Float	Target's entry price at detection time
target_leverage	Integer	
reason_skipped	String	e.g. "drift_too_large", "portfolio_exposure_too_large"

Column	Type	Notes
detected_at	DateTime	When the ghost was created
last_seen_at	DateTime	Last time a fill/close event was received for this symbol

web_trades

Audit trail of every fill (opens, reduces, closes, seeds).

Column	Type	Notes
fill_id	String UNIQUE	HL's tid or composite fallback key
wallet_addr	String	
symbol	String	
side	String	LONG / SHORT
direction	String	e.g. "Open Long" , "Close Short" , "> Long"
size	Float	Our position size (coin quantity)
price	Float	Post-slippage execution price
notional	Float	size × price
leverage	Integer	
realized_pnl	Float	Null on opens; written by db_record_close()
fee	Float	Taker fee deducted from balance
is_seed	Boolean	True = startup seeded position
is_debounced	Boolean	True = entered via HFT debounce filter
target_entry_px	Float	Target's original fill price (calibration — null for non-debounced)
copy_delay_ms	Float	Ms between target fill and our copy entry (calibration)
timestamp	DateTime	

web_equity

Periodic equity snapshots (every ~30s). Used for charting and analytics.

Column	Type	Notes
wallet_addr	String	
equity	Float	balance + margin + unrealized_pnl
balance	Float	Free cash (no margin locked)

Column	Type	Notes
upnl	Float	Sum of unrealized PnL across all open positions
timestamp	DateTime	

4. Session Lifecycle

4.1 Session creation

`_create_session(address, label, start_balance, copy_mode, debounce_secs, detected_style)` in `sim.py` :

- Instantiates `WalletMonitor` , `TradeExecutor` , `PositionSizer` , `HyperliquidClient`
- Creates `WalletSession` dataclass with all fields at defaults
- Registers in `_sessions[address]`

4.2 Session start — `start_session(session, emit_fn, offset_secs)`

This is the async initialization sequence. It runs entirely inside the background asyncio loop.

Step 1 — Restart recovery

Load `db_load_positions(address)` and `db_get_latest_equity_snapshot(address)` .

- **Full restore** (saved positions + snapshot exist): restore `simulated_balance` from snapshot's `balance` field (free cash). Restore `simulated_positions` with original entry prices and copy ratios. Restore counters (`wins` , `losses` , `pnl` , `fees`) from `db_restore_session_counters()` . No re-seeding — double-deducting margin would corrupt the balance.
- **Balance-only restore** (snapshot exists, no positions): restore `simulated_balance` from snapshot's `equity` field (the wallet had all positions closed cleanly). Re-seeding will run next.
- **Fresh start** (no snapshot): use `start_balance` . Re-seeding will run next.

Step 2 — Fetch target state

`monitor.get_current_state()` makes a REST call to `GET /info` → `clearinghouseState` for the target address. Returns `UserState` with balance, positions, and orders.

Step 3 — Compute copy_ratio

```
copy_ratio = session.start_balance / target.balance
```

This is locked for the session. It never changes during trading (only updated in the periodic snapshot for display purposes, guarded by `_state_lock`).

Step 4 — Ghost reconciliation (restart case)

Load persisted `ghost_positions` from DB. Cross-check each ghost against the live target state:

- Ghost no longer held by target → remove from ghost state and DB
- Ghost still held → update `last_seen_at` and `target_size`

This prevents re-evaluating positions that were already decided on in a prior run.

Step 5 — Startup seeding

Only runs when there are no saved positions (fresh start or balance-only restore). See [Section 6](#) for the full engine.

Step 6 — Historical fills and style detection

`_fetch_target_fills(session, limit=50)` pulls the last 50 fills from the target wallet across all 9 HL sub-DEXes. These seed the `web_trades` DB table (so the trades API has data from day one) and populate `session.recent_fills` for the UI feed.

`_detect_trading_style(session)` classifies the target as HFT or Swing (see [Section 5](#)).

Step 7 — Initial equity snapshot

Computes `equity = balance + margin + upnl` and writes to DB + `equity_history`.

Step 8 — Callbacks and monitoring

Wires the five event callbacks (`on_new_position`, `on_position_close`, `on_position_update`, `on_new_order`, `on_order_fill`) plus `on_leverage_change` to `session.monitor`. Starts `_periodic_equity_snapshot` as an async task. Starts `monitor.start_monitoring()` which opens the WebSocket and begins the listener.

4.3 Session reset — `_reinit_session`

Triggered by the "Reset" button in the dashboard. Clears all in-memory state (positions, ghosts, PnL counters, balance back to `start_balance`). Calls `purge_wallet_data()` to wipe the DB. Re-runs the full seeding sequence using the same `evaluate_startup_position()` decision engine (not the old naive loop). Re-runs `_detect_trading_style()`.

4.4 Session removal

`DELETE /api/remove-wallet/<address>` : stops monitoring (`monitor.stop()`), removes from `_sessions`, deletes from `wallets` table, optionally purges trade data.

5. Trading Style Auto-Detection

`_detect_trading_style(session)` is an async function called at session start, after reset, and every 6 hours by the periodic snapshot task.

5.1 Fill rate measurement

Fetches up to 500 fills from the primary DEX for the target wallet (no DB side-effects — only timestamps are needed for classification):

```
fills_per_hour = len(raw_fills) / span_hours
# span_hours = (newest_fill_time - oldest_fill_time) / 3,600,000 ms
# clamped to minimum 1.0 so a wallet with all fills in one burst doesn't get infinity
```

5.2 Median hold time computation

Pairs "Open" fills with subsequent "Close" / "Reduce" fills for the same coin using FIFO matching:

```
for each fill in time order:
    if "Open" in direction:
        open_times[coin].append(fill_time)
    elif "Close"/"Reduce" in direction and open_times[coin] is non-empty:
        hold_ms = fill_time - open_times[coin].pop(0)
        holds_ms.append(hold_ms)

median_hold_secs = median(holds_ms) / 1000
```

If fewer than 2 fills exist, the function returns without changing mode. `session.median_hold_secs` defaults to `60.0`.

5.3 Classification decision

```
if fills_per_hour >= settings.copy_style.hft_threshold_fills_per_hour: # default 60
    copy_mode      = "debounced"
    debounce_secs  = clamp(median_hold_secs × 0.25, min=10, max=300)
    detected_style = "HFT"
else:
    copy_mode      = "all_fills"
    detected_style = "Swing"
```

The adaptive debounce logic: debounce at 25% of the median hold duration. A target with a 2-minute median hold gets a 30-second debounce; a 10-minute holder gets 150 seconds. The floor of 10s prevents noise-chasing; the ceiling of 300s keeps positions actionable.

5.4 Persistence and periodic re-check

The detected style, copy mode, and debounce seconds are written back to the `wallets` DB table via `db_update_wallet_style()` so restarts load the last-known classification without re-fetching.

The periodic snapshot task fires a re-check every 6 hours:

```
if time.monotonic() - session._style_last_checked > 21_600:
    asyncio.create_task(_detect_trading_style(session))
```

6. Startup Seeding — Ghost Positions & Decision Engine

6.1 The core invariant

A position we did not explicitly open must **NEVER** generate an execution order. This is enforced by:

1. The startup decision engine (which decides whether to open or ghost each position)
2. The `ghost_positions` dict (which intercepts all events for ghosted symbols)
3. Hard assertions before every order placement: `assert symbol not in session.ghost_positions`

6.2 `evaluate_startup_position()` — pure decision function

Takes a target `Position`, current mark price, portfolio state, and policy. Returns `(decision, seed_size, reason)`. No I/O, no side effects.

Checks are applied in strict order. The first failing check returns `GHOST_ONLY` :

#	Check	Threshold	Returns on fail
1	Missing entry price	<code>entry_price <= 0</code>	<code>GHOST_ONLY("missing_entry_price")</code>
2	Entry drift	<code>abs(mark - entry) / entry > 1.5%</code>	<code>GHOST_ONLY("drift_too_large")</code>
3	Leverage cap	—	Cap to <code>min(pos.leverage, max_seed_leverage=4)</code> , do not ghost
4	Compute seed size	<code>size × copy_ratio × 0.35</code>	—
5	Per-position exposure	<code>seed_notional / equity > 3%</code>	<code>GHOST_ONLY("position_exposure_too_large")</code>
6	Portfolio total exposure	<code>(total_copied + seed) / equity > 25%</code>	<code>GHOST_ONLY("portfolio_exposure_too_large")</code>
7	Symbol concentration	<code>(symbol_notional + seed) / equity > 10%</code>	<code>GHOST_ONLY("symbol_exposure_too_large")</code>
8	Daily loss guard	<code>daily_loss_pct >= 3%</code>	<code>GHOST_ONLY("daily_loss_guard")</code>

#	Check	Threshold	Returns on fail
9	Drawdown guard	<code>drawdown_pct >= 10%</code>	<code>GHOST_ONLY("drawdown_guard")</code>
10	Soft checks	<code>drift > 1% OR leverage > 3</code>	<code>SEED_SMALL (50% of seed_size) or GHOST_ONLY("soft_risk_reduced_and_skipped")</code>
—	Dust guard	<code>seed_notional < \$10</code>	<code>GHOST_ONLY("below_dust_guard")</code>
—	All pass	—	<code>SEED_NOW</code>

startup_mode overrides:

- `"always_skip"` : immediately return `GHOST_ONLY` without running checks
- `"always_seed"` : run checks but override `GHOST_ONLY` decisions except `missing_entry_price`
- `"smart_safe"` : run checks as defined above (default)

6.3 Seeding execution

For `SEED_NOW` and `SEED_SMALL` :

- Use `pos.entry_price` (target's actual entry) as the simulated entry price — not current mark price. This makes the simulation reflect the target's full position history from their original entry.
- Leverage is capped at `min(pos.leverage, max_seed_leverage)` then adjusted via `position_sizer.calculate_leverage()`
- Compute `margin = seed_size × mark_price / leverage`
- Charge seed fee: `seed_size × mark_price × taker_fee_rate`
- Deduct `margin + fee` from `simulated_balance`
- Write to `simulated_positions` and `web_trades` (with `is_seed=True`)
- Tag `seeded_on_startup: True` on the position dict

For `GHOST_ONLY` :

- Add to `session.ghost_positions[symbol]` with `{side, target_size, target_entry_price, target_leverage, reason_skipped, detected_at, last_seen_at}`
- Write to `ghost_positions` DB table

A startup summary is logged after all positions are evaluated:

```
[Wallet] Startup seeding: SEED_NOW=3 | SEED_SMALL=1 | GHOST=2 reasons={'drift_too_large': 2}
```

6.4 Ghost position event handling

When any fill or close event arrives:

1. Check if `symbol` in `session.ghost_positions` — before any processing
2. If yes: update `target_size` , `last_seen_at` ; persist to DB; return early — **no order**

3. For close/reduce: remove from ghost state and DB (position gone from target)
4. For flip (">"): remove ghost, allow the open-side fill to flow through as a fresh live copy event
5. If not a ghost: assert `symbol not in ghost_positions` (belt-and-suspenders), then proceed normally

7. Fill Processing Pipeline

7.1 WebSocket event flow

```
HL WebSocket → WalletMonitor._handle_user_event()
    → _handle_fills(fills[]) → on_order_fill() per fill
    → _handle_positions(positions[]) → patches current_state + on_position_close() if size=0
```

`_handle_fills` runs **before** `_handle_positions` in the same WS message. This means `current_state` still holds the pre-close size when the fill handler reads it for close-fraction calculation.

7.2 Fill deduplication

Every fill has a `tid` (trade ID). If `tid` is absent (rare), a composite key (`coin`, `px`, `sz`, `dir`) is used. All processed fill IDs are stored in `session._processed_fill_ids`: dict (ordered, insertion-order preserved in Python 3.7+).

Eviction: When the dict exceeds 100,000 entries, the oldest 50,000 are deleted. This is a sliding window — recent fills stay deduplicated even after eviction. The old approach (`.clear()`) was dangerous for HFT because it wiped all dedup state, allowing reconnect replays to reprocess fills.

7.3 on_order_fill — the dispatch gate

The closure in `make_callbacks()` . Handles only the **routing** layer:

1. Check session alive and not paused
2. Dedup check
3. Parse `symbol`, `direction`, `target_size`, `price`
4. Blocked-asset check
5. **Ghost guard** — if `symbol` is in `ghost_positions`, update ghost state and return
6. Parse direction flags: `is_closing`, `is_flip`, `is_opening`
7. **Mode dispatch:** if `copy_mode == "debounced"` and `is_opening` and not `is_flip` → schedule debounce task, return
8. Call `await _process_fill(session, fill_data, fill_id, emit_fn)`

7.4 _process_fill — the execution engine

This is the module-level function called by both the direct path and the debounce path.

Step 1 — Parse fill metadata

```
symbol, side_str, target_size, price, direction = from fill_data
_target_px    = fill_data.get("_target_px")      # injected by debounce task
_fill_time_ms = fill_data.get("_fill_time", 0)   # injected by debounce task
_is_debounced = fill_data.get("_debounced", False)
_copy_delay_ms = (now_ms - _fill_time_ms) if _fill_time_ms else None
```

Step 2 — Scale size

```
our_size      = target_size × session.copy_ratio
our_notional  = our_size × price    (pre-slippage, used for dust guard only)
```

Step 3 — Dust guard

If `our_notional < $10:increment skipped_fills_count`, return. The \$10 threshold matches HL's real minimum order notional so the simulation's skip behaviour mirrors live trading.

Step 4 — Slippage model

Applied to `price` before fee and position calculations:

```
slippage = settings.sim_accuracy.slippage_bps / 10_000 # default 0.0003

# Opening a LONG → we chase the price up
if is_opening and not is_flip:
    price = price × (1 + slippage) if LONG
    price = price × (1 - slippage) if SHORT

# Closing a LONG → we get filled lower
if is_closing and not is_flip:
    existing_side = simulated_positions[symbol]["side"]
    price = price × (1 - slippage) if existing_side == LONG
    price = price × (1 + slippage) if existing_side == SHORT
```

For flips the slippage is applied once in the direction of the new open (adverse for the old close too — both are unfavorable in the same direction, which is correct).

Step 5 — Latency drift (all_fills mode only)

```
lat_ms = settings.sim_accuracy.sim_latency_ms # default 150
if lat_ms > 0 and is_opening and not is_flip and copy_mode == "all_fills":
    drift_pct = (lat_ms / 1000) × 0.0001 × uniform(0.5, 1.5)
    price = price × (1 + drift_pct) if LONG
    price = price × (1 - drift_pct) if SHORT
```

The 0.0001 constant is a rough volatility proxy: 10bps of drift per second, randomised $\pm 50\%$ to avoid deterministic bias. Debounced positions skip this because the mark-price re-pricing at debounce fire already incorporates real price movement.

Step 6 — Position count guard

```
if is_opening and not is_flip:
    if max_open_trades and len(simulated_positions) >= max_open_trades:
        skipped_fills_count += 1; return
```

Step 7 — Net exposure guard

```
if is_opening and not is_flip:
    long_notional = sum(p["value"] for p in positions where side==LONG)
    short_notional = sum(p["value"] for p in positions where side==SHORT)
    equity = balance + total_margin + upnl
    new_long/short = long/short + our_notional (for the relevant side)
    if abs(new_long - new_short) / equity > max_net_exposure_pct (0.80):
        skipped_fills_count += 1; return
```

Step 8 — Fee calculation

```
is_flip_check = ">" in direction
if is_flip_check and symbol in simulated_positions:
    # Flip: pay for closing the old position + opening the new one
    old_sz = abs(simulated_positions[symbol]["size"])
    fill_fee = (old_sz + our_size) * price * taker_fee_rate
else:
    # Single open or close: post-slippage price
    fill_fee = (our_size * price) * taker_fee_rate * (2 if is_flip_check else 1)
```

Note: closes re-compute the fee inside the lock (using `close_size` and optionally maker rate) and apply a `fee_delta` correction. The pre-charge here is an estimate corrected below.

Step 9 — Leverage lookup

```
target_leverage = find pos.leverage from monitor.current_state.positions where symbol matches
our_leverage = position_sizer.calculate_leverage(
    target_leverage * adjustment_ratio, # default 0.5x
    capped at min(per_asset_max, max_leverage)
)
```

Per-asset leverage caps are hardcoded in `PositionSizer._MAX_LEVERAGE` (e.g., BTC: 50x, SOL: 20x, unknown assets: 10x).

Step 10 — Lock acquisition and state mutation

async with session._state_lock: — all state changes happen atomically.

Close / Reduce path:

```
fraction = min(target_size / target_pos_size, 1.0) # same fraction target closed
close_size = our_pos_size × fraction

# Maker fee on some closes (configurable, default 0%)
fee_rate = taker_fee_rate if random() > maker_close_rate else taker_fee_rate × (3.5/4.5)
close_fee = close_size × price × fee_rate
fee_delta = close_fee - fill_fee # correct the pre-charge
balance -= fee_delta
fees_paid += fee_delta

pnl = close_size × (price - entry) if LONG
pnl = close_size × (entry - price) if SHORT

margin_returned = pos["margin_used"] × fraction
balance += margin_returned + pnl
simulated_pnl += pnl
wins/losses += 1

# Update or delete position
if remaining_size < 1e-8:
    del simulated_positions[symbol]
    db_delete_position(...)
else:
    pos["size"] -= close_size
    pos["margin_used"] -= margin_returned
    pos["value"] = remaining_size × entry
    db_upsert_position(...)

db_record_fill(..., is_debounced, target_entry_px, copy_delay_ms)
db_record_close(..., pnl)
```

Open / Add / Flip path:

For a flip, the existing opposite-side position is closed first at the fill price with its own PnL calculation before the new side is opened.

```

position_value = our_size × price          # post-slippage
margin_req     = position_value / max(our_leverage, 1)

# VWAP entry price for adds
old_notional   = abs(pos["size"]) × pos["entry_price"]
new_size       = abs(pos["size"]) + our_size
new_entry      = (old_notional + position_value) / new_size    # weighted average

pos["entry_price"] = new_entry
pos["size"]        = ±new_size
pos["value"]       += position_value
pos["margin_used"] += margin_req
pos["leverage"]    = our_leverage
balance -= margin_req

db_upsert_position(...)
db_record_fill(..., is_debounced, target_entry_px, copy_delay_ms)

```

Step 11 — Emit (outside lock)

```

equity = balance + total_margin + upnl
session.equity_history.append({"t": ..., "equity": ..., "balance": ..., "upnl": ...})
# capped at 2000 entries

emit_fn("fill",          {symbol, direction, size, price, notional, pnl, ...})
emit_fn("state_update",  _session_to_dict(session))
emit_fn("equity_tick",   {wallet, equity})

```

8. Simulation Accuracy Models

8.1 Slippage

Applied to every fill in `_process_fill` (Step 4 above) and in `on_position_close` (safety net).

Direction convention:

- Opening LONG → price moves **up** (you chase the ask)
- Opening SHORT → price moves **down**
- Closing LONG → price moves **down** (you hit the bid)
- Closing SHORT → price moves **up**

Default: 3 bps (0.03%) per side. For 5,000 fills/day this represents ~0.3% daily drag (3bps × 2 sides × 5000 trades / 365 ≈ meaningful over time).

8.2 Execution latency drift (all_fills mode)

Approximates the price drift during the window between the WebSocket event arriving and the order being acknowledged by the exchange (~100–300ms in practice):

```
drift_pct = (latency_ms / 1000) × 0.0001 × uniform(0.5, 1.5)
```

At 150ms default: $0.15 \times 0.0001 \times \sim 1.0 = 0.0015\%$ (1.5 bps) mean drift per trade. Applied only to opens, not closes (a close order is always urgent and doesn't chase price in the same way).

Debounce mode skips this because the debounce re-prices at current mark, which already captures however far the price moved during the debounce window.

8.3 Taker fee model

Default: 4.5 bps (HL Tier 0). Deducted from `simulated_balance` on every fill. For closes, the `maker_close_rate` config (default 0) allows a fraction of closes to be charged at the maker rate (3.5 bps). This models strategies that place reduce-only limit orders rather than market orders.

Fee is charged on post-slippage notional. This is correct: your actual fee in live trading is based on the price you actually filled at.

8.4 Funding charges

Fetches via `get_funding_rates()` every 35s (shared across all wallets via `_funding_cache`). Applied in `_periodic_equity_snapshot` every ~30s:

```
charge = position_value × funding_rate / 120
# 120 = number of 30s periods per hour
# Longs pay charge; shorts receive charge
balance -= charge (LONG)
balance += charge (SHORT)
```

HL's funding rate field is the 1-hour predicted rate (not 8-hour). Pro-rating by 120 converts it to a per-30s tick. The actual HL settlement is once per hour on the hour — this pro-rating is accurate on average but slightly off for positions closed within the first hour.

8.5 Liquidation price

The periodic snapshot uses HL's actual `liquidationPx` from WebSocket position updates (already available in `monitor.current_state.positions[i].liquidation_price`), scaled to our leverage:

```
lev_ratio = our_leverage / target_leverage
liq_px = target_liq_px × lev_ratio + entry_price × (1 - lev_ratio)
```

Intuition: if the target holds 10x and their liq is at entry × 0.90, and we hold 5x, our liq is further away. The linear interpolation between `target_liq_px` and `entry_price` using our leverage ratio gives a good approximation.

Fallback when target's `liq_px` is unavailable:

```
maintenance = 1.0 / (2.0 × leverage)
liq_px = entry × (1 - 1/leverage + maintenance) # LONG
liq_px = entry × (1 + 1/leverage - maintenance) # SHORT
```

The computed `liq_px` is stored in `pos["_liq_px"]` so the UI display is consistent with the check that would actually trigger liquidation.

8.6 Close-fraction accuracy

When the target closes partially, we close the same fraction of **our** position:

```
fraction = min(target_close_size / target_pre_close_size, 1.0)
close_size = our_position_size × fraction
```

`target_pre_close_size` is snapshotted from `monitor.current_state` **before** the state lock, because `_handle_positions` (which would update `current_state`) runs after `_handle_fills` in the same WS message. This guarantees the snapshot reflects the size before the close.

9. Risk Management Guards

9.1 Dust guard (\$10 minimum notional)

Any fill where `our_size × price < $10` is skipped and counted toward `skipped_fills_count`. This matches HL's real exchange minimum, so the simulation's skip behaviour mirrors what would happen in live trading.

9.2 Blocked assets

Any fill where `symbol.upper()` in `settings.copy_rules.blocked_assets` is silently skipped. Configured as a comma-separated environment variable (`BLOCKED_ASSETS=BTC,ETH`).

9.3 Position count cap

```
if len(simulated_positions) >= settings.copy_rules.max_open_trades:
    skip fill
```

Default: unlimited. Enforced at fill time, not at seeding time.

9.4 Net exposure guard

Prevents the portfolio from becoming highly directional. Checked before every opening fill:

```
|long_notional - short_notional| / equity > 80% → skip
```

The equity denominator uses live `_upnl()` so it adapts to unrealized moves.

9.5 Daily loss cap

```
_daily_loss_usd += abs(pnl) # on every losing close
if _daily_loss_usd >= max_daily_loss_usd:
    session.is_paused = True
```

Resets at midnight UTC. Default: \$500. When triggered, the session is paused and a Telegram alert is sent.

9.6 Circuit breaker (fast drawdown)

Checked in `_periodic_equity_snapshot` after computing `equity` :

```
window_ago_isostring = (now - timedelta(seconds=fast_loss_window_secs)).isoformat()
baseline = most recent equity_history entry at or before window_ago
if (baseline - equity) / baseline >= fast_loss_pct:
    session.is_paused = True
```

Default: 5% equity drop within 5 minutes. Captures rapid losses from leverage that the per-trade daily cap would miss.

10. HFT Debounce System

10.1 Why debounce exists

An HFT fill arrives on the WebSocket. By the time a live copy bot acknowledges the fill, transmits an order, and receives exchange confirmation, 100–500ms have elapsed. An HFT trade that lasts 2 seconds has already reversed. Copying it would be buying into a reversal. Debounce solves this by only copying positions that survive long enough to be "real" rather than noise.

10.2 Scheduling a debounced copy

When `on_order_fill` detects `copy_mode == "debounced"` and an opening fill arrives:

```
_schedule_debounced_copy(session, fill_data, fill_id, emit_fn)
```

`_pending_debounce[symbol]` is set to:

```
{
    "task":          asyncio.Task (the timer coroutine),
    "total_sz":      target_size (accumulates with each Add fill),
    "fill_template": fill_data (carries direction/side/coin metadata),
    "original_px":   price at the time of the first fill,
    "original_time": fill_data["time"] (ms timestamp)
}
```

The `fill_id` is immediately added to `_processed_fill_ids` so reconnect replays don't re-trigger the debounce.

10.3 Accumulating Add fills during the window

If an "Add" fill arrives for a symbol that is already pending debounce:

1. Cancel the existing timer task
2. `pending["total_sz"] += new_target_size`
3. Create a new timer task (resets the debounce window from the latest signal)
4. Replace `pending["task"]`

This ensures the debounce fires with the full accumulated position size, not just the first open's size.

10.4 Debounce task execution

When the timer fires after `delay_secs` :

1. Pop from `_pending_debounce`
2. Check `session.address` in `_sessions` and not `session.is_paused`
3. Check target still holds the position: `any(p.symbol == symbol for p in monitor.current_state.positions)` . If not → `log "closed within N seconds"` and return. No position is opened.
4. Fetch current mark price: `await _get_shared_mids(session.client)` — TTL 3s for accuracy
5. Build patched fill:

```
patched = {
    **fill_template,
    "px":          str(current_mark_price),    # re-price at current market
    "sz":          str(total_sz),              # full accumulated size
    "_target_px":  original_px,                # for calibration stats
    "_fill_time":  original_time_ms,          # for calibration stats
    "_debounced": True,
}
```

6. Temporarily set `session.copy_mode = "all_fills"` to avoid re-triggering debounce
7. Call `await _process_fill(session, patched, fill_id, emit_fn)`

The re-pricing at current mark is what makes the simulation realistic: it shows the price you would actually have received if you'd waited N seconds before placing your order.

10.5 Close fills during debounce

Close fills for a pending-debounce symbol bypass the debounce check entirely:

```
if copy_mode == "debounced" and is_opening and not is_flip: # ← only opening
    → schedule debounce
```

If the target closes a position before our debounce fires, the close fill reaches `_process_fill`, finds symbol not in `simulated_positions`, and returns via the "untracked position" guard. The debounce task will then fire, find the position gone, and also return. Net result: no position is opened or closed. Correct.

11. Leverage Change Tracking

`WalletMonitor._handle_positions()` processes the `positions` field of every `userEvents` WS message. It patches `monitor.current_state.positions` in-place with fresh data. When it detects a change in leverage for an existing position:

```
if abs(new_leverage - old_leverage) > 0.01:
    asyncio.create_task(self.on_leverage_change(symbol, old_leverage, new_leverage))
```

The `on_leverage_change` callback in `make_callbacks()`:

```
our_new_lev = position_sizer.calculate_leverage(new_lev, adjustment_ratio, ...)
pos_value = abs(pos["size"]) × pos["entry_price"]
old_margin = pos["margin_used"]
new_margin = pos_value / max(our_new_lev, 1)
margin_delta = new_margin - old_margin

async with session._state_lock:
    pos["leverage"] = our_new_lev
    pos["margin_used"] = new_margin
    balance -= margin_delta # positive delta = more margin locked
    db_upsert_position(...)
```

Effect: if the target reduces their leverage (e.g., 10x → 5x), their margin requirement increases. We follow the same direction but at our scaled leverage. If our leverage was already at the cap (`max_seed_leverage`), it will stay capped.

12. Periodic Tasks

12.1 `_periodic_equity_snapshot` — every 25–35s

Runs as an async task for each wallet. The random jitter (25 + uniform(0, 10)) prevents all wallets from calling the API simultaneously.

Sequence:

1. **copy_ratio refresh** — re-compute from target's current balance inside `_state_lock` :

```
new_ratio = start_balance / monitor.current_state.balance
async with _state_lock:
    session.copy_ratio = new_ratio
```

This keeps the copy ratio in sync with the target's balance over weeks of trading.

2. **Price map** — use `monitor.current_state.positions[i].current_price` (from last WS patch). REST fallback via `_get_shared_mids()` for any symbols not in the current state.
3. **Unrealized PnL** for each position:

```
upnl = size × (mark - entry)  LONG
upnl = size × (entry - mark)  SHORT
```

4. **Funding charges** — for each position, look up funding rate from cache, pro-rate:

```
charge = abs(size) × mark_price × funding_rate / 120
balance -= charge  (LONG)
balance += charge  (SHORT)
total_funding_paid += charge  (LONG)
total_funding_paid -= charge  (SHORT)  # earns funding
```

5. **Equity computation:**

```
equity = simulated_balance + total_margin + total_upnl
```

6. **Circuit breaker check** — compare equity against baseline from `fast_loss_window_secs` ago in `equity_history` .
7. **Liquidation check** — for each position, compute `liq_px` (from HL's WS data or fallback formula), store in `pos["_liq_px"]` , then check:

```
LONG liquidated if mark <= liq_px
SHORT liquidated if mark >= liq_px
```

On liquidation: realize PnL at `liq_px`, delete position, emit `"margin_call"` event. If equity drops to ≤ 0 , pause session and emit `"liquidated"` .

8. **Style re-check** — if 6 hours have elapsed since last check:

```
asyncio.create_task(_detect_trading_style(session)) .
```

9. **Snapshot write** — append to `equity_history` (in-memory, capped at 2000 entries) and write to `web_equity` DB table.

10. **Emit** — `state_update` to UI.

12.2 `WalletMonitor._periodic_state_refresh` — every 50–70s

Calls `get_current_state()` (REST) to keep `monitor.current_state` fresh independent of fill events. This prevents the fill handler from reading stale leverage values for symbols with no recent WS activity. Wider jitter (0–30s initial stagger) keeps 15 wallets spread across a 30s window.

12.3 Reconnect fill-gap recovery

When the WebSocket disconnects and reconnects, `_replay_missed_fills()` runs:

- 1. Fetch all fills for the target since `_last_fill_time` (the timestamp of the last processed fill) across all 9 sub-DEXes
- 2. Deduplicate by `tid`
- 3. **Cap at 500 fills** — if > 500, keep only the 500 most recent. This prevents HFT accounts from flooding the pipeline after a reconnect
- 4. Sort by time and replay through `_handle_fills()`

13. Analytics & Stats Engine

`web/stats.py` — pure functions over DB rows. No side effects.

13.1 Core metrics

Metric	Formula
Win rate	<code>wins / (wins + losses) × 100</code>
Profit factor	<code>sum(positive PnLs) / abs(sum(negative PnLs))</code>
Expectancy	<code>avg_win × win_rate + avg_loss × loss_rate</code> (per trade)
Max drawdown	<code>(trough - peak) / peak × 100</code> over all equity snapshots
Calmar ratio	<code>total_return_pct / abs(max_drawdown_pct)</code>
Sharpe ratio	<code>mean(daily returns) / stdev(daily returns) × √365</code>
Annualized return	<code>((1 + total_return/100) ^ (365/days_running) - 1) × 100</code>
Volatility	<code>stdev(daily returns) × √365 × 100</code> (annualized %)

13.2 Composite quality score (0–100)

Weights four normalized sub-scores:

```

score = sharpe_score × 0.35
      + calmar_score × 0.25
      + win_rate_score × 0.20
      + consistency_score × 0.20

```

```

sharpe_score:      normalized from [-2, 3]
calmar_score:      normalized from [-1, 5]
win_rate_score:    normalized from [30%, 70%]
consistency_score: inverse of rolling Sharpe volatility (stability)

```

13.3 Automated COPY / MONITOR / SKIP decision

Evaluated after the score is computed:

SKIP (immediate disqualifier, checked first):

- Max drawdown < −40%
- Score < 35
- Score < 55 AND performance declining week-over-week

COPY (all five conditions must pass):

- Score ≥ 65
- Sharpe ≥ 0.5
- Max drawdown ≥ −30%
- Closed trades ≥ 20
- Profitable days ≥ 40%

MONITOR (promising, not ready):

- Score ≥ 45 but not all COPY conditions met

INSUFFICIENT DATA: < 5 completed round-trip trades.

13.4 Capital bracket calculator

Given the smallest opening notional observed in the simulation and the current copy_ratio, computes minimum required real capital at three thresholds:

```

min_capital      = $10  × start_balance / smallest_sim_open_notional  (HL dust guard)
suggested_capital= $50  × start_balance / smallest_sim_open_notional
optimal_capital  = $100 × start_balance / smallest_sim_open_notional

```

This answers: "how much real money do I need to copy this account without every small trade hitting HL's minimum order size?"

13.5 Rolling metrics

- **Rolling 50-trade win rate:** $O(n)$ sliding window, sampled to max 500 output points
- **Rolling 7-day Sharpe:** daily equity returns over a sliding 7-day window
- **PnL histogram:** closed PnLs bucketed into 20 equal-width bins

13.6 Fee drag analysis

```
total_fees           = sum of all taker fees (opens + closes + seeds)
total_volume         = sum of notionals for live (non-seed) fills
fee_pct_vol          = total_fees / total_volume × 100      (% of traded volume)
fee_drag_pct         = total_fees / gross_pnl × 100         (% of gross profit eaten by fees)
breakeven_notional   = $0.10 / taker_fee_rate              (minimum trade to cover single fill fee)
```

14. REST API Endpoints

All endpoints in `web_app.py` :

Method	Path	Description
GET	<code>/api/wallets</code>	List all sessions with live state
POST	<code>/api/add-wallet</code>	Add wallet {address, label, start_balance}
POST	<code>/api/remove-wallet/<addr></code>	Stop monitoring, remove from DB
POST	<code>/api/pause/<addr></code>	Pause fill copying
POST	<code>/api/resume/<addr></code>	Resume fill copying
POST	<code>/api/reset/<addr></code>	Full reset (purge DB, re-seed)
GET	<code>/api/state/<addr></code>	Live session dict (positions, equity, PnL)
GET	<code>/api/equity/<addr>?hours=N</code>	Equity history for chart
GET	<code>/api/trades/<addr>?from=&to=</code>	Trade records (date-filterable)
GET	<code>/api/stats/<addr></code>	Full analytics + COPY/MONITOR/SKIP decision
GET	<code>/api/calibration/<addr></code>	HFT calibration stats (entry slippage, delay)
GET	<code>/api/export/trades/<addr></code>	CSV export of all trades
GET	<code>/api/export/equity/<addr></code>	CSV export of equity snapshots

WebSocket events emitted to clients:

Event	Payload	Trigger
state_update	Full session dict	Every fill, every snapshot tick
fill	{symbol, direction, size, price, notional, pnl}	Every processed fill
equity_tick	{wallet, t, equity}	Every snapshot tick and every fill
position_close	{wallet, symbol, pnl}	Safety-net close
margin_call	{wallet, symbol, liq_price, pnl}	Liquidation simulated
liquidated	{wallet, equity}	Equity reaches ≤ 0

15. HFT Calibration System

15.1 What is recorded

Every trade that goes through the debounce path has three extra fields written to `web_trades` :

Field	Value
is_debounced	True
target_entry_px	The price in the target's original fill (before our 30s wait)
copy_delay_ms	<code>(time.time() × 1000) - original_fill_time_ms</code> — milliseconds elapsed

15.2 The calibration endpoint

GET `/api/calibration/<wallet>` calls `db_get_hft_calibration_stats()` :

```
{
  "debounced_trades": 847,
  "closed_trades": 612,
  "win_rate_pct": 54.2,
  "avg_entry_slippage_pct": 0.18,
  "avg_copy_delay_ms": 31420,
  "median_copy_delay_ms": 30150
}
```

`avg_entry_slippage_pct` is the key metric:

- Positive = we entered at a worse price than the target (we paid more for longs / sold lower for shorts)
- Negative = we entered at a better price (the position moved in our favour during the debounce window)

Over time, this tells you whether the debounce threshold is tuned correctly. If `avg_entry_slippage_pct` is consistently positive and large, the debounce window is too long (you're always chasing the move). If it's consistently negative (but

win rate is low), the target's trades reverse after 30s and you're capturing the reversal — debounce is doing its job but the underlying strategy may not be worth copying.

15.3 Closing the 12% accuracy gap

The current simulation accuracy for HFT wallets is ~88%. The remaining 12% gap is empirical — it can only be measured by running a real small-money live bot alongside the simulation and comparing outcomes. The calibration data makes this comparison possible:

- 1. Run simulation for 60+ days on an HFT wallet
- 2. Call `/api/calibration/<wallet>` to get `avg_entry_slippage_pct` and `median_copy_delay_ms`
- 3. Update `settings.sim_accuracy.slippage_bps` to match the measured slippage
- 4. Consider adjusting `hft_debounce_secs` if `win_rate_pct` for debounced trades is substantially different from the overall wallet win rate

16. Accuracy & Production Readiness

16.1 What the simulation correctly models

Mechanic	Accuracy
Taker fees (all fills, post-slippage price)	Exact
VWAP entry price on position adds	Exact
Close fraction from target's pre-close size	Exact
Flip PnL (close old side + open new side)	Exact
Funding charges (pro-rated from live rates)	~98% (exact timing would need hourly settlement)
Slippage (opens and closes, safety-net closes)	Configurable; default 3bps/side
Execution latency drift (all_fills mode)	Approximation (~1.5bps at 150ms)
Leverage change mid-position	Exact (from WS events)
Liquidation (HL's actual <code>liquidationPx</code> scaled)	High fidelity
Circuit breaker, daily loss cap, position cap	Exact
Net exposure guard	Exact
HFT debounce re-pricing	Exact (re-prices at ≤ 3 s-old mark)
Style detection re-check every 6h	Adaptive

16.2 Remaining simulation gaps

Gap	Impact
Funding settlement exact timing	Low (pro-rated vs hourly)
<code>pos["value"]</code> uses entry price not current mark	Low (net exposure guard sees slightly stale notional)
Constant slippage (not per-asset)	Low for liquid assets; could be >3bps for small caps
Partial fills on large orders	Low at copy-ratio-scaled sizes
HFT debounce quality uncertainty	~12% empirical gap (see Section 15)

16.3 Simulation accuracy scores

Wallet type	Accuracy
Long-term / Swing	~95%
HFT (debounced)	~88%

16.4 What is needed for live trading (current gaps)

The simulation is production-grade. To send real orders requires five additional components:

1. **executor.py** — implement real HL `/exchange` POST with EIP-712 signed payloads (`eth_account` or `hyperliquid-python-sdk`). Currently returns fake order IDs.
2. **updateLeverage call** — before every new-symbol opening order, call the leverage-set action. Without this, HL uses whatever leverage was last set on that symbol in your account (default can be 20x or 50x).
3. **Own-wallet fill confirmation** — subscribe to your own `userEvents` WebSocket feed to confirm orders actually filled before updating position state. Currently only the target's fills are tracked.
4. **reduceOnly: true** on all close/reduce orders — prevents accidentally opening a new position if the size calculation is slightly off.
5. **Per-asset minimum lot size** — HL enforces minimum coin quantities per asset (e.g., 0.001 BTC). The current dust guard checks \$10 notional but doesn't round to the correct decimal places. Orders below HL's minimum are silently rejected.